



IGNITION IO

User Guide

The **Crucible** Workbench and the **Igni** CLI

Set up a chip-design toolchain, run digital and analog experiments,
and reproduce the exact same environment on any machine — without
installing a single tool by hand.

What's in this guide

Read sections 1 to 3 once. After that, jump straight to the flow you need — digital, synthesis or analog. Every command in this guide is copy-paste ready.

01	Before You Begin	What the product is, in plain words	3
02	Setup in Five Minutes	From a blank machine to a working lab	5
03	Command Reference	Every igny command, with examples	7
04	Digital Flow — Verilog Simulation	Compile, simulate, view waveforms	9
05	Synthesis Flow — RTL to Gate Netlist	Yosys with a standard-cell library	12
06	Analog Flow — SPICE and Python	Ngspice decks and design-equation checks	14
07	Sharing and Reproducing Work	Lock, sync, and hand off to a teammate	16
08	Troubleshooting	Common messages and what to do	17
09	Quick Reference Card	One page, every command	19

WHO THIS GUIDE IS FOR

No prior experience with EDA tools is assumed. If you have never opened a waveform viewer or run a SPICE deck before, follow the guide top to bottom and you will have working results by the end of section 4. If you already know these tools, section 3 and section 9 are all you need.

SECTION 01

Before You Begin

What Crucible does, and the three words you need to understand it.

1.1 The problem this solves

Chip design work needs a stack of tools — a simulator, a waveform viewer, a synthesis engine, a SPICE engine, a cell library. Normally you install each one by hand, match versions by hand, and fix the PATH by hand. It breaks often, and it almost never works the same way on a colleague's machine.

Crucible is a managed workbench. It installs those tools for you, keeps each version tied to a named project, and runs them for you. Nothing is installed globally on your system, so nothing clashes with software you already have.

Ignity is the command you type. It is the command-line interface to Crucible. Every action in this guide — creating a project, installing a tool, running a simulation — starts with `ignity`.

1.2 Three words to learn

Everything in Crucible is built out of these three ideas. Learn them once and the rest of the guide reads easily.

WORD	THINK OF IT AS	WHAT IT ACTUALLY DOES
Environment	A named toolbox	A labelled set of tools and their exact versions. You create one per project, for example <code>verilog-lab</code> . Tools get installed into an environment.
Workspace	Your project folder	A folder on disk that is attached to one environment. It contains a <code>crucible.toml</code> file. When you run a tool from inside this folder, Crucible knows which toolbox to open.
Tool	The actual program	An EDA program from the Crucible catalog — Icarus Verilog, GTKWave, Yosys, Ngspice, and others. You install it once and it is downloaded into a shared inventory on your machine.

THE ONE RULE TO REMEMBER

Always run `ignity` commands from the **workspace root** — the folder that contains `crucible.toml`. If you are one folder too deep or one folder too high, Crucible cannot tell which environment you mean, and the command will fail.

1.3 Tools available to you

These are the catalog tools used throughout this guide. There are more in the catalog; these five cover the digital, synthesis and analog flows.

INSTALL NAME	WHAT IT IS	USED FOR
iverilog	Icarus Verilog	Compiles your Verilog design and testbench into a runnable simulation. Also provides the <code>vvp</code> runner, so you do not install <code>vvp</code> separately.
gtkwave	Waveform viewer	Opens the <code>.vcd</code> file a simulation produces, so you can see signals as a timing diagram.
yosys	Synthesis engine	Turns behavioural RTL into a gate-level netlist mapped to real standard cells.
ngspice	Analog simulator	Runs SPICE circuit decks — DC operating points, transient waveforms, transistor-level analysis.
matplotlib	Python plotting	Used by the Python analysis scripts to draw design-verification plots.

1.4 What you need on your machine

Required

- The Igny CLI installed and on your PATH
- An internet connection for the first tool install
- A terminal (Command Prompt, PowerShell, or a shell)
- A text editor for writing Verilog and SPICE files

Not required

- Any pre-installed EDA tool
- Administrator rights for tool installs
- Manual PATH edits or environment variables
- A licence server

Examples in this guide use Windows-style paths (`.\build\spice.log`). On macOS or Linux, use forward slashes (`./build/spice.log`) and replace `type` with `cat`. Every `igny` command itself is identical on all platforms.

SECTION 02

Setup in Five Minutes

Do this once. Every experiment afterwards reuses it.

1 Start Crucible

Run this once on a new machine. It prepares Crucible on your system.

```
ign y init
```

2 Create an environment

Pick any name you like. This is the toolbox your project will use.

```
ign y env create verilog-lab
```

3 Create a workspace and go into it

`--path` is the folder to create. `--env` attaches it to the environment from step 2.

```
ign y workspace create --path ./my-lab --env verilog-lab  
cd ./my-lab
```

4 Install the tools you need

Each tool downloads once into a shared inventory and is bound to `verilog-lab`. Install only what your work needs — you can add more later at any time.

```
ign y tool install iverilog  
ign y tool install gtkwave  
ign y tool install yosys      # only if you will synthesize  
ign y tool install ngspice   # only if you will run analog
```

5 Lock the toolset

This freezes the exact versions you just installed so the project stays reproducible. Run it after every install.

```
ign y workspace lock
```

WHAT YOU SHOULD SEE

```
C:\Users\you\Desktop> igny env create verilog-lab
Environment 'verilog-lab' created (stack=default)
C:\Users\you\Desktop> igny workspace create --path ./my-lab --env verilog-lab
Workspace 'my-lab' created (env=verilog-lab)
C:\Users\you\Desktop\my-lab> igny tool install iverilog
✓ Installed iverilog
  Source: artifact
  Path:   C:\Users\you\Crucible\tools\iverilog\...
  Bound to env: verilog-lab
```

CHECK IT WORKED

The install prints the version and the path it was placed at. A tick and a Bound to env line means the tool is ready. Nothing was added to your system PATH — that is intentional, and it is why `igny run` exists.

2.1 How to organise your files

Keep one folder per experiment inside the workspace. Run each experiment from its own folder.

```
my-lab/
├─ crucible.toml           ← the workspace marker, do not delete
├─ 01_dff/
│   ├─ dff.v               ← your design
│   └─ dff_tb.v            ← your testbench
├─ 02_counter/
└─ 03_amplifier/
    ├─ scripts/run_analysis.py
    ├─ spice/design.cir
    └─ build/               ← logs and outputs land here
```

SECTION 03

Command Reference

Every command you will use, grouped by what it is for.

SETTING UP

COMMAND	WHAT IT DOES
<code>ign y init</code>	Prepares Crucible on this machine. Run once.
<code>ign y env create <name></code>	Creates a named environment (a toolbox).
<code>ign y workspace create --path <folder> --env <name></code>	Creates a project folder and attaches it to an environment. Creates <code>crucible.toml</code> inside it.

MANAGING TOOLS

COMMAND	WHAT IT DOES
<code>ign y tool install <tool></code>	Downloads a catalog tool once and binds it to the current environment. Prints the version and install path.
<code>ign y workspace lock</code>	Freezes the current tool versions into the workspace, so they can be restored exactly later.
<code>ign y workspace sync</code>	Reads the lock and installs those exact versions. This is how a teammate reproduces your setup.

RUNNING TOOLS

COMMAND	WHAT IT DOES
<code>ign y run <tool> <arguments></code>	Runs a tool from the environment. Everything after the tool name is passed to the tool exactly as written.
<code>ign y run script <file.py></code>	Runs a Python script inside the environment, with the environment's Python packages available.

HOW IGNY RUN WORKS

Think of `ign y run` as a prefix. If a tool's own documentation tells you to type `yosys -s script.tcl`, you type `ign y run yosys -s script.tcl`. Ign y does not rewrite or interpret those arguments — it hands them straight to the tool. This means every tutorial and manual for these tools still applies.

One tool can provide several commands. Installing `iverilog` also gives you `vvp`, so `ign y run vvp ...` works with no separate install.

3.1 Reading a command

3.2 Common options you will meet

OPTION	BELONGS TO	MEANING
-o <file>	iverilog	Name of the compiled simulation to produce.
-s <module>	iverilog	Which module is the top level. Use your testbench module name.
-s <file>	yosys	Run the commands in this script file. (Same letter, different tool, different meaning.)
-b	ngspice	Batch mode. Runs and exits without opening a window. Use for DC checks.
-o <file>	ngspice	Write the simulation log to this file.

SECTION 04

Digital Flow — Verilog Simulation

Write a design, test it, and look at the waveform. This is the flow you will use most.

4.1 The three commands

Every digital experiment is the same three steps, in the same order.

STEP	COMMAND	WHAT HAPPENS
Compile	<code>igny run iverilog -o sim.vvp -s tb_module design.v tb.v</code>	Checks your Verilog and builds a runnable simulation file.
Simulate	<code>igny run vvp sim.vvp</code>	Runs it. Prints the testbench log and writes a <code>.vcd</code> waveform file.
View waves	<code>igny run gtkwave tb.vcd</code>	Opens the waveform viewer so you can see the signals over time.

WHAT THE FILE TYPES MEAN

`.v` is your Verilog source. `.vvp` is the compiled simulation — a build output, safe to delete and regenerate. `.vcd` is a Value Change Dump: a recording of every signal at every moment, which is what the waveform viewer draws.

4.2 Worked example — D flip-flop

A D flip-flop is the simplest storage element in digital design: on each rising clock edge it copies the input to the output and holds it. The reset here is synchronous and active-high, meaning it only clears the output on a clock edge, never instantly.

THE DESIGN — DFF.V

```
module dff (
    input wire d_in, clock, reset,
    output reg Q_out, Qb_out
);
always @(posedge clock) begin
    if (reset) begin Q_out ≤ 1'b0; Qb_out ≤ 1'b1; end
    else          begin Q_out ≤ d_in; Qb_out ≤ ~d_in; end
end
endmodule
```

RUN IT

```
cd 01_dff
igny run iverilog -o dff_sim.vvp -s dff_tb dff.v dff_tb.v
igny run vvp dff_sim.vvp
igny run gtkwave dff_tb.vcd
```

WHAT YOU SHOULD SEE

```
VCD info: dumpfile dff_tb.vcd opened for output.
Time=0      | reset=1 | d_in=0 | Q_out=x | Qb_out=x
Time=5000   | reset=1 | d_in=0 | Q_out=0 | Qb_out=1
Time=16000  | reset=0 | d_in=0 | Q_out=0 | Qb_out=1
Time=30000  | reset=0 | d_in=1 | Q_out=0 | Qb_out=1
Time=35000  | reset=0 | d_in=1 | Q_out=1 | Qb_out=0
DFF test passed.
dff_tb.v:59: $finish called at 86000 (1ps)
```

Read it like this: `Q_out` follows `d_in` one clock later, and `Qb_out` is always the opposite of `Q_out`. Both start as `x` (unknown) and only settle after the first clocked reset — which is exactly what "synchronous reset" means.

4.3 How to tell pass from fail

Testbenches in this lab follow one convention, so you always read the result the same way.

YOU SEE	IT MEANS
<code>... test passed.</code>	PASS. Every check matched and the run reached <code>\$finish</code> normally.
<code>ERROR: ...</code>	FAIL. A check did not match. The testbench stops at the first mismatch, so the line printed is the exact failure — fix that one and re-run.
<code>\$finish called at ...</code>	The simulation ended cleanly at that time. On its own this is not a pass — look for the pass line above it.

A testbench compares your design against a known-correct reference and prints an error the moment they disagree. That is why a passing run is short and quiet: no news is good news.

4.4 GTKWave in sixty seconds

GTKWave opens with an empty waveform area — this is normal, you choose what to display.

- Pick your testbench module in the top-left **SST** panel.
- Select signals in the panel below it and click **Append** to add them to the view.
- Use **Zoom Fit** to see the whole simulation at once, then zoom into the region you care about.
- Multi-bit signals such as `count[3:0]` show as value boxes; right-click to switch between hex, decimal and binary.

HARMLESS MESSAGES

On some systems GTKWave prints lines such as `GLIB_OLD_LOG_API` or a `gdk_pixbuf` assertion warning when it starts. These are cosmetic warnings from the graphics library. If the window opens and your signals are there, the tool is working correctly.

4.5 More designs to try

Same three commands, different files. Each of these is a standard teaching example and a good way to check your setup end to end.

DESIGN	TYPE	WHAT IT DEMONSTRATES
Controlled up/down counter	Sequential	A 4-bit counter that counts up by 2 or down by 1 depending on a mode input, and wraps at its limits.
Seven-segment decoder	Combinational	Maps each of the 16 hex inputs to a fixed 7-bit display pattern. Pure logic — no clock involved.
Mealy "101" detector	State machine	Watches a serial bit stream and flags every occurrence of the pattern 101, including overlapping ones.
Three-number adder	Combinational	Adds three 8-bit numbers into a 10-bit result, sized so the sum can never overflow.

```
# the pattern for any of them
ignpy run iverilog -o sim.vvp -s <testbench_module> <design>.v <design>_tb.v
ignpy run vvp sim.vvp
ignpy run gtkwave <design>_tb.vcd
```

SECTION 05

Synthesis Flow — RTL to Gate Netlist

Turn the Verilog you wrote into the actual logic gates a chip would be built from.

5.1 What synthesis does

The Verilog you write describes behaviour — "on each clock edge, store this value". Synthesis converts that description into real cells: an `assign` becomes gate logic, and an `always @(posedge clk)` becomes a D flip-flop. The output is a netlist: the same design written purely as a list of standard cells and the wires between them.

The cells come from a **standard-cell library** — a file describing the gates a real manufacturing process offers. This lab uses the open Nangate 45 nm library (`NangateOpenCellLibrary_typical.lib`).

5.2 The synthesis script

Yosys is driven by a script so the flow is repeatable. Save these commands as `yosys_commands.tcl`. Each line is one stage of the conversion.

#	COMMAND	WHAT IT DOES
1	<code>read_verilog top.v</code>	Reads your RTL source.
2	<code>hierarchy -check -top top</code>	Works out the module hierarchy and sets the top module.
3	<code>proc</code>	Converts <code>always</code> , <code>if</code> and <code>case</code> blocks into multiplexers and registers.
4	<code>techmap</code>	Maps everything to Yosys's own generic gate library first.
5	<code>dfflibmap -liberty ...Nangate...lib</code>	Replaces generic flip-flops with real Nangate flip-flop cells.
6	<code>abc -liberty ...Nangate...lib</code>	Optimises the combinational logic and maps it to Nangate gates.
7	<code>clean</code>	Removes unused cells and dangling wires.
8	<code>write_verilog -noattr synth_example.v</code>	Writes out the finished gate-level netlist.

5.3 Run it

```
ign y run yosys -s yosys_commands.tcl
```

WHAT YOU SHOULD SEE

```
-- Executing script file `yosys_commands.tcl' --
7.2. Executing DEMUXMAP pass.
Dumping module `top'.

End of script. Logfile hash: dce466e6ac
Yosys 0.63+190
Time spent: 4% 3x read_verilog (0 sec), 4% 2x write_verilog (0 sec), ...
```

HOW TO CHECK THE RESULT

Yosys prints statistics after each stage. The important one comes after `dfflibmap` and `abc`: it reports the **cell count and area** of the mapped design. Then open `synth_example.v` — it should contain only Nangate cell instances and wires, with none of your original `always` or `assign` lines left. That is how you know synthesis actually happened.

5.4 A minimal design to try it on

A 2-to-1 multiplexer feeding a register. After synthesis this should become one D flip-flop plus the handful of gates that implement the multiplexer.

```
module top(a, b, clk, select, out);
  input  a, b, clk, select;
  output out;
  reg    out;
  wire   y;

  assign y = (select) ? b : a;    // 2:1 multiplexer → gate logic
  always @(posedge clk)
    out ≤ y;                      // register → D flip-flop
endmodule
```

SECTION 06

Analog Flow — SPICE and Python

Transistor-level circuits, checked two ways: by equation and by simulation.

6.1 The two checks

Analog experiments pair a small Python script with a SPICE deck. Running both is the point: if the maths and the simulator agree, your design is sound.

The Python check

Works out the design equations — sizing, bias points, gain, expected frequency — and prints a `PASS` line. It never touches the simulator, so it is an independent opinion.

The SPICE check

Simulates the actual circuit in Ngspice and reports node voltages and currents. This is the physics answer, and it should land close to the equation answer.

6.2 Setup and run

```
# one time, for the whole analog lab
igny env create analog_experiments
igny workspace create --path analog_demo --env analog_experiments
cd analog_demo
igny tool install ngspice
igny tool install matplotlib

# per experiment, run from that experiment's folder
igny run script .\scripts\run_analysis.py
igny run ngspice -b -o build/spice.log spice/design.cir
type .\build\spice.log
```

BATCH MODE OR WINDOW MODE

Use `-b` for DC operating-point runs — the simulation finishes and writes everything to the log file, no window needed. **Drop the `-b`** when the deck contains a transient analysis you want to watch: without it, the deck's `.control` block opens the live waveform in the Ngspice window. If your plot never appears, you probably left `-b` on.

6.3 Reading the output

The Python script prints its numbers and a pass line:

```
$ igny run script .\scripts\run_analysis.py
gm=1.00 mS  ro=125.0 kOhm  gain=-6.90 V/V
VoutQ=1.05 V  pole=230.8 MHz
PASS common_source_amplifier
```

The SPICE log confirms it from the simulator's side:

```
$ igny run ngspice -b -o build/spice.log spice/design.cir
Circuit: Common Source Amplifier
Doing analysis at TEMP = 27.000000 and TNOM = 27.000000
Using SPARSE 1.3 as Direct Linear Solver

No. of Data Rows : 1
v(out) = 1.300673e+00
Tool 'ngspice' completed successfully
```

`v(out)` is the voltage at the node named `out` in your deck. `i(vd)` would be the current through the source named `vd`. A negative current simply reflects SPICE's sign convention for current flowing into a voltage source — it is not an error.

6.4 Experiments you can build up through

These run in order of difficulty: device modelling first, then logic, then amplifiers, then an oscillator.

#	EXPERIMENT	WHAT IT CHECKS
01	MOSFET parameter extraction	Oxide capacitance and process-gain values for a first-order MOS process.
02	Short-channel nMOSFET bias	Device width for a target drain current, and how far it sits from leaving saturation.
03	CMOS & pseudo-nMOS NAND	Same logic function in two styles, and how the low output level differs between them.
04	Common-source amplifier	Bias point, small-signal gain, output headroom, dominant pole.
05	CMOS differential amplifier	Differential gain, common-mode gain, CMRR, input common-mode range.
06	Op-amp square-wave generator	A relaxation oscillator above 1 MHz, watched live as a transient run.

IF THE TWO CHECKS DISAGREE

A gap between the equation result and the SPICE result is information, not a bug. It usually means an assumption in the hand calculation does not hold — a device left saturation, or a second-order effect the first-order equation ignores. Check the bias point first: that is where the two normally part company.

SECTION 07

Sharing and Reproducing Work

The reason for all of this: someone else gets your exact toolchain, first try.

7.1 Lock, then share

When you lock a workspace, Crucible records the exact version of every tool bound to it. That record lives in the workspace folder, so it travels with your code.

```
# you: freeze and publish
ignpy workspace lock
git add . && git commit -m "lab setup" && git push
```

7.2 Reproduce on another machine

Your teammate needs two commands. No tool installs, no version hunting, no PATH edits.

```
# them: clone and restore
git clone <repo>
cd <repo>
ignpy workspace sync
```

`ignpy workspace sync` reads the lock and installs precisely those versions. They now have the same simulator, the same waveform viewer, the same synthesis engine you used. If your results reproduce, it is because the toolchain genuinely is identical — not because it happened to be close enough.

GOOD HABITS

Commit `crucible.toml` and the lock file. They are the whole point.

Do not commit build outputs — `.vvp`, `.vcd`, and the `build/` folder are regenerated in seconds and only bloat the repository.

Re-run `ignpy workspace lock` whenever you install a new tool, so the lock never falls behind what you are actually using.

SECTION 08

Troubleshooting

What the message means, and the one thing to try first.

WHAT YOU SEE	WHY	FIX
Tool not found, or the command is not recognised	The tool is not installed in this environment, or you typed the tool name without <code>ign run</code> .	Run <code>ign tool install <tool></code> , then retry with the <code>ign run</code> prefix.
Not a workspace / no environment found	You are not in the folder that contains <code>crucible.toml</code> .	<code>cd</code> back to the workspace root and run the command from there.
iverilog reports no top-level module, or picks the wrong one	Icarus cannot tell which module starts the simulation.	Add <code>-s <testbench_module></code> — use the module name inside the testbench file, not the filename.
Syntax error at a line number when compiling	Verilog syntax problem in that file.	Fix the first error listed and recompile. Later errors are often knock-on effects of the first.
ERROR: followed by expected and got values	The testbench found a mismatch and stopped there.	This is a design bug, not a tool problem. The printed line tells you which case failed — fix that behaviour and re-run.
GTKWave opens but the waveform area is empty	No signals have been added to the view yet.	Pick the module in the SST panel, select signals, click Append, then Zoom Fit.
GTKWave prints <code>GLIB_OLD_LOG_API</code> or a <code>pixbuf</code> assertion	Cosmetic warning from the graphics library.	Ignore it. If the window opened, the tool is fine.
No <code>.vcd</code> file was produced	The testbench does not dump waveforms.	The testbench needs <code>\$dumpfile("name.vcd")</code> and <code>\$dumpvars</code> . Without them nothing is recorded.

WHAT YOU SEE	WHY	FIX
Ngspice runs but no plot window appears	You used <code>-b</code> , which is batch mode.	Drop <code>-b</code> for transient runs that should open a window. Keep it for DC operating-point runs.
Ngspice log has values but they look wrong	Node names in the deck do not match what you expect.	Check that <code>v(out)</code> refers to the node your circuit actually names <code>out</code> . Node names in SPICE are exactly as written in the deck.
Yosys stops at <code>hierarchy -check</code>	A module in the design is missing or misnamed.	Make sure every module the top level instantiates is in a file passed to <code>read_verilog</code> .
A teammate's <code>sync</code> installs nothing	The lock file was never committed, or is out of date.	Run <code>ign workspace lock</code> , commit the result, and have them pull and sync again.
File not found on a path you typed	Slash direction or a wrong relative path.	Windows uses <code>.</code> <code>\build\spice.log</code> ; macOS and Linux use <code>./build/spice.log</code> . Paths are relative to the folder you are standing in.

STILL STUCK?

Copy the full command you ran and the complete output — not just the last line — and send it to the Ignition team. The lines before the error usually say more than the error itself.

SECTION 09

Quick Reference Card

Everything on one page. Print it, or keep it open beside your terminal.

SETUP — ONCE PER PROJECT

<code>ign y init</code>	Start Crucible on this machine
<code>ign y env create <env></code>	Create a toolbox
<code>ign y workspace create --path <dir> --env <env></code>	Create a project folder, attach it
<code>ign y tool install <tool></code>	Add a tool to the environment
<code>ign y workspace lock</code>	Freeze versions
<code>ign y workspace sync</code>	Restore frozen versions

DIGITAL — VERILOG SIMULATION

<code>ign y run iverilog -o sim.vvp -s tb_mod design.v tb.v</code>	Compile
<code>ign y run vvp sim.vvp</code>	Simulate, write <code>.vcd</code>
<code>ign y run gtkwave tb.vcd</code>	View waveforms

SYNTHESIS — RTL TO GATES

<code>ign y run yosys -s yosys_commands.tcl</code>	Synthesize to a gate-level netlist
--	------------------------------------

ANALOG — SPICE AND PYTHON

<code>ign y run script .\scripts\run_analysis.py</code>	Equation check
<code>ign y run ngspice -b -o build/spice.log spice/design.cir</code>	DC run, batch mode
<code>ign y run ngspice -o build/spice.log spice/design.cir</code>	Transient run, opens a window
<code>type .\build\spice.log</code>	Read the log (<code>cat</code> on macOS/Linux)

Three things that fix most problems

- Run from the workspace root — the folder with `crucible.toml`.
- Prefix every tool with `ign y run`; nothing is on your system PATH by design.
- Read the first error, not the last. The rest usually follow from it.

Getting help

Send the full command and its complete output to the Ignition team.

ignition.io